

第4章

数据库安全性

在第1章中已经讲到,数据库的特点之一是由数据库管理系统提供统一的数据保护功能来保证数据的安全可靠和正确有效。数据库的数据保护主要包括数据库的安全性和完整性。本章讲解数据库的安全性,第5章将讨论数据库的完整性。

本章导读

数据库的安全性问题和计算机系统的安全性是紧密联系的,计算机系统的安全性问题可大致分为三大类,即技术安全类、管理安全类和政策法律类(详见二维码内容)。



扩展阅读: 计算机系统的三类安全性问题

本章讨论数据库技术安全类问题,即从技术上如何保证数据库系统的安全性。

本章主要介绍有关计算机信息安全和数据库安全标准的内容。从4.2节起讲解数据库安全性控制的主要技术。希望读者理解和掌握通过SQL语句实现自主存取控制权限的授予和收回的方法;掌握创建用户、角色,以及分配权限给角色的方法。要加强实验练习掌握这些方法,还要进一步理解什么是强制存取控制机制,如何利用视图技术、审计技术、数据存储加密和传输加密等技术手段提高数据库的安全性。

本章较难理解的内容是强制存取控制机制中确定主体能否存取客体的存取规则,要理解制定该存取规则的原因。

4.1 数据库安全性概述

数据库的安全性是指保护数据库,以防不合法使用所造成的数据泄露、篡改或破坏。

安全性问题不是数据库系统所独有的,所有计算机系统都存在不安全因素,只是在数据库系统中由于大量数据集中存放,而且为众多用户直接共享,数据成为一个部门、企业乃至一个国家重要的资源,从而使安全性问题更为突出。系统安全保护措施是否有效是数据库系统的主要技术指标之一。

4.1.1 数据库的不安全因素

数据库的安全性事故频发。例如,2016年初某地多所医院连续遭受勒索软件的侵害,给医院和患者带来了极大的不便,对医院的工作造成了极大的负面影响。

综合来看,对数据库安全性产生威胁的因素主要有以下几方面。

1. 非授权用户对数据库的恶意存取和破坏

一些黑客(hacker)或不法分子在用户存取数据库时窃取用户名和用户口令等信息,然后假冒合法用户获取、修改甚至破坏用户数据。有的黑客还故意锁定并修改数据,进行勒索和破坏等犯罪活动。因此,必须阻止有损数据库安全的非法操作,以保证数据免受未经授权的访问和破坏。

2. 数据库中重要或敏感的数据被泄露

黑客或敌对分子千方百计地盗窃数据库中重要或敏感的机密数据,造成数据泄露。例如,攻击者利用SQL注入(SQL injection)技术,在应用程序中事先定义好的查询语句的结尾添加额外的SQL语句,在入侵检测不严的情况下欺骗数据库服务器执行非授权的查询和操作,进行SQL注入攻击,使得机密数据被泄露、篡改或锁定,数据库数据被破坏。

3. 安全环境的脆弱性

数据库的安全性与计算机系统(包括计算机硬件、操作系统、网络系统等)的安全性是紧密联系的。操作系统安全性脆弱、网络协议安全性保障不足等都会造成数据库安全性的破坏。因此,必须加强计算机系统整体的安全性保护。随着互联网、云计算和大数据技术的发展,计算机系统的安全性问题越来越突出。为此,必须建立一套可信(trusted)计算机系统的概念和标准。只有建立了完善的可信标准,即安全标准,才能规范和指导安全计算机系统部件的生产,较为准确地测定产品的安全性能指标,满足民用和军用的不同需要。

4.1.2 安全标准简介

计算机以及信息安全技术方面有一系列安全标准,最有影响的是可信计算机系统评价标准TCSEC和工厂安全通用准则CC标准。

1. 可信计算机系统评价标准 TCSEC

1985年美国国防部(Department of Defense, DoD)正式颁布了《可信计算机系统评价标准》(trusted computer system evaluation criteria,简称TCSEC或DoD85)。1991年的TCSEC/Trusted Database Interpretation(TCSEC/TDI)又把TCSEC扩展到了数据库系统。

TCSEC/TDI根据计算机系统对各项指标的支持情况将系统划分为4组7个等级,依次是D, C(C1、C2), B(B1、B2、B3), A(A1),其系统可靠或可信程度逐渐增高,如表4.1所示。

表 4.1 TCSEC/TDI 安全级别划分

安全级别	安全指标
A1	验证设计(verified design)
B3	安全域(security domain)
B2	结构化保护(structural protection)
B1	标记安全保护(labeled security protection)
C2	受控的存取保护(controlled access protection)
C1	自主安全保护(discretionary security protection)
D	最小保护(minimal protection)

D 级：该级是最低级别。保留 D 级的目的是将一切不符合更高标准的系统都归于 D 组。

C1 级：该级只提供了非常初级的自主安全保护，能够实现对用户和数据的分离，进行自主存取控制(discretionary access control, DAC)，保护或限制用户权限的传播。

C2 级：该级是安全产品的最低档，提供受控的存取保护，即将 C1 级的 DAC 进一步细化，以个人身份注册负责，并实施审计和资源隔离。

B1 级：标记安全保护。对系统的数据加以标记，并对标记的主体和客体实施强制存取控制(mandatory access control, MAC)以及审计等安全机制。B1 级别的产品才被认为是真正意义上的安全产品，满足此级别的产品前一般多冠以“安全”(security)或“可信的”字样，作为区别于普通产品的安全产品出售。

B2 级：结构化保护。建立形式化的安全策略模型，并对系统内的所有主体和客体实施 DAC 和 MAC。

B3 级：安全域。该级的可信计算基(trusted computing base, TCB)必须满足访问监控器的要求，审计跟踪能力更强，并提供系统恢复过程。

A1 级：验证设计，即在提供 B3 级保护的同时，给出系统的形式化设计说明和验证，以确信各安全保护真正实现。

表 4.2 列出了不同安全级别对安全指标的支持情况。随着安全级别的由低到高，系统对各项安全指标的支持也从无到有。其中，□表示该级不提供对该指标的支持，■表示该级新增的对该指标的支持，▒表示该级对该指标的支持与相邻低一级的等级一样，▨表示该级对该指标的支持较下一级有所增加或改动。

TCSEC/TDI 从 4 个方面来描述安全性级别划分的指标，即安全策略、责任、保证和文档。每个方面又细分为若干项，如表 4.2 中第一、二行所示。

表 4.2 不同安全级别对安全指标的支持情况

安全指标	安全策略							责任			保证								文档				
	自主存取控制	客体重用	标记完整性	标记信息的扩散	主体敏感度标记	设备标记	强制存取控制	标识与鉴别	可信路径	审计	系统体系结构	系统完整性	屏蔽信道分析	可信设施管理	可信恢复	可安全测试	设计规范和验证	配置管理	可信分配	安全特性用户指南	可信设施手册	测试文档	设计文档
C1	■	□	□	□	□	□	□	■	□	□	■	■	□	□	□	■	□	□	□	■	■	■	■
C2	▨	■	□	□	□	□	□	▨	□	■	■	■	□	□	□	▨	□	□	□	■	▨	■	■
B1	■	■	■	■	□	□	■	▨	□	▨	▨	■	□	□	□	▨	■	□	□	■	▨	■	▨
B2	■	■	■	■	■	■	▨	■	■	▨	▨	■	■	■	□	▨	▨	■	■	■	▨	■	▨
B3	▨	■	■	■	■	■	■	▨	▨	▨	■	■	▨	▨	■	▨	▨	■	□	■	▨	■	▨
A1	■	■	■	■	■	■	■	■	■	■	■	■	▨	■	■	▨	▨	▨	■	■	■	■	■

2. IT 安全通用准则 CC

自 1993 年起,多个国家组织了专门的委员会开发 IT 安全通用准则 (common criteria, CC)。经过多次讨论和修订后,CC 2.1 版于 1999 年被 ISO 采用为国际标准,2001 年被我国采用作为国家标准。有关国际和我国信息安全标准发展简述,请参见二维码内容。



扩展阅读:信息安全标准发展简述

根据系统对安全保证要求的支持情况,CC 提出了评估保证级 (evaluation assurance level, EAL),从 EAL1 至 EAL7 共分为 7 级,按保证程度逐渐增高。粗略而言,CC 的评估保证级与 TCSEC/TDI 的安全级别大致对应如表 4.3 所示。

表 4.3 CC 评估保证级 (EAL) 的划分及与 TCSEC/TDI 安全级别的对应

评估保证级	定义	TCSEC/TDI 安全级别 (近似相当)
EAL1	功能测试 (functionally tested)	
EAL2	结构测试 (structurally tested)	C1
EAL3	系统地测试和检查 (methodically tested and checked)	C2
EAL4	系统地设计、测试和复查 (methodically designed, tested and reviewed)	B1
EAL5	半形式化设计和测试 (semiformally designed and tested)	B2
EAL6	半形式化验证的设计和测试 (semiformally verified design and tested)	B3
EAL7	形式化验证的设计和测试 (formally verified design and tested)	A1

4.2 数据库安全性控制

在计算机系统中,安全措施是一级一级层层设置的。例如,在图 4.1 所示的安全模型中,用户要求进入计算机系统时,系统首先根据输入的用户标识进行用户身份鉴别,只有合法的用户才准许进入计算机系统。对已进入系统的用户,数据库管理系统要执行安全保护,采取多种存取控制机制,目的是只允许用户执行合法操作,在用户退出系统后,根据情况还要进行安全审计。操作系统也会有自己的保护措施,读者可参考操作系统的有关书籍。对敏感数据,在传输过程中要进行加密保护,最后还应以密码形式存储到数据库中。

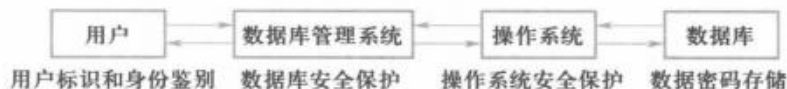


图 4.1 计算机系统的安全模型

本章讨论与数据库有关的安全性技术,对于强力逼迫透露口令、盗窃物理存储设备等行为而采取的保安措施,例如出入机房登记、加锁等,不在讨论之列。

图 4.2 是数据库管理系统的安全性控制模型示例。主要包含以下几部分:

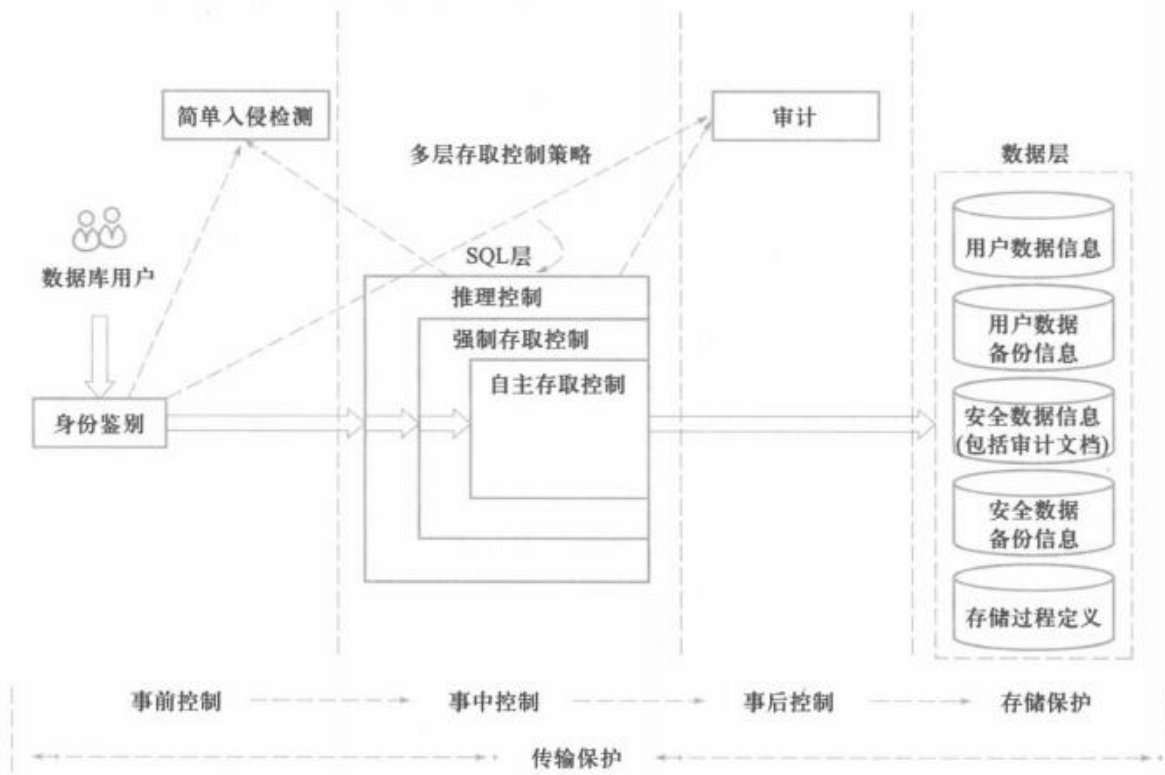


图 4.2 数据库管理系统的安全性控制模型

① 事前控制。通过身份鉴别和简单入侵检测共同实现事前的访问保护。

② 事中控制。数据库管理系统在 SQL 层提供多层存取控制策略,包括自主存取控制、强制存取控制、推理控制等。用户只能按照被授予的权限和安全规则存取合法数据。

③ 事后控制。为监控恶意访问,可根据具体安全需求配置审计规则,对用户访问行为和系统关键操作进行审计,及时发现可疑攻击者,把有关信息返回入侵检测子系统,对异常用户行为进行处理和阻拦。

④ 存储保护。在数据层,对敏感数据、重要的存储过程定义加密存储。数据库管理系统不仅存放用户数据,还存储与安全有关的审计文档,并进行加密存储。

⑤ 传输保护。用户身份信息、SQL 语句和数据等频繁地在客户端与服务器端进行传输,若采用明文方式传输很容易被截获或篡改,存在安全隐患。为了保证这些信息能够安全地进行交换和传输,数据库管理系统提供了传输加密功能。

4.2.1 用户身份鉴别

用户身份鉴别是数据库管理系统提供的最外层安全保护措施。每个用户在系统中都有一个用户标识。每个用户标识由用户名(user name)和用户标识号(user identification number, UID)两部分组成。UID 在系统的整个生命周期内是唯一的。系统内部记录着所有合法用户的标识,系统鉴别是指由系统提供一定的方式让用户标识自己的名字或身份。每次用户要求进入系统时,由系统进行核对,通过鉴定后才提供使用数据库管理系统的权限。

用户身份鉴别的方法有很多种,而且在一个系统中往往也是多种方法结合,以获得更强的安全性。常用的用户身份鉴别方法主要有以下几种。

1. 静态口令鉴别

静态口令一般由用户自己设定,鉴别时只要按要求输入正确的密码,系统即可允许用户使用数据库管理系统。这些密码是静态不变的,方式简单,但容易被攻击,安全性较低。因此通常采用双因子鉴别,即口令+数字证书。提供一套口令策略,包括密码复杂度、密码过期时限、登录失败用户锁定、密码历史保存等。在存储和传输过程中密码信息不可见,均以密文方式存在。

用户身份鉴别可以重复多次。

2. 动态口令鉴别

这种方式的口令是动态变化的,每次鉴别时均需使用动态产生的新口令登录数据库管理系统,即采用一次一密的方法。常用的方式如验证码和动态令牌方式,每次鉴别时要求用户使用验证码或动态令牌登录数据库管理系统。与静态口令鉴别相比,这种认证方式的安全性相对高一些。

3. 生物特征鉴别

它是一种通过生物特征进行认证的技术,其中,生物特征是指生物唯一具有的可测量、识别和验证的稳定生物特征,如指纹、虹膜、掌纹和人脸识别等。这种方式通过采用图像处理和

模式识别等技术实现了基于生物特征的认证,与传统的口令鉴别相比,其安全性无疑较高。

4. 智能卡鉴别

智能卡是一种不可复制的硬件,内置集成电路的芯片,具有硬件加密功能。智能卡由用户随身携带,登录数据库管理系统时将智能卡插入专用的读卡器进行身份验证。由于每次从智能卡中读取的数据是静态的,通过内存扫描或网络监听等技术还是可能截取到用户的身份验证信息,存在安全隐患。因此,实际应用中一般采用个人识别号(personal identification number, PIN)和智能卡相结合的方式。这样,即使PIN或智能卡中有一种被窃取,用户身份仍不会被冒充。

5. 入侵检测

检测前,管理员应按实际需求定义检测规则。在检测过程中,依据预先设置的检测规则实时进行入侵分析。若发现入侵情况,实时进行处理。

处理方式包括通过邮件报警和断开会话连接、锁定用户进行处罚等。

4.2.2 存取控制

数据库安全控制最重要的一点就是,确保只授权给有资格的用户访问数据库的权限,同时令所有未被授权的人员无法接近数据。这主要通过数据库系统的存取控制机制实现。

存取控制机制主要包括定义用户权限和合法权限检查两部分。

① 定义用户权限,并将用户权限登记到数据字典中。用户对某一数据对象的操作权力称为权限。某个用户应该具有何种权限是管理和政策方面的问题,而不是技术问题。数据库管理系统的功能是保证这些决定的执行。为此,数据库管理系统必须提供适当的语言来定义用户权限,这些定义经过编译后存储在数据字典中,被称作安全规则或授权规则。

② 合法权限检查。每当用户发出存取数据库的操作请求后(请求一般应包括操作类型、操作对象和操作用户等信息),数据库管理系统查找数据字典,根据安全规则进行合法权限检查,若用户的操作请求超出了定义的权限,系统将拒绝执行此操作。

定义用户权限和合法权限检查机制一起组成了数据库管理系统的存取控制子系统。

C2级的数据库管理系统支持自主存取控制,B1级的数据库管理系统支持强制存取控制。

这两类方法的简单定义是:

① 在自主存取控制方法中,用户对于不同的数据库对象有不同的存取权限,不同的用户对同一对象也有不同的权限,而且用户还可将其拥有的存取权限转授给其他用户。因此自主存取控制非常灵活。

② 在强制存取控制方法中,每一个数据库对象被标以一定的密级,每一个用户也被授予某一个级别的许可证。对于任意一个对象,只有具有合法许可证的用户才可以存取。强制存取控制因此相对比较严格。

下面介绍这两种存取控制方法。

4.2.3 自主存取控制方法

大型数据库管理系统都支持自主存取控制, SQL 标准也对自主存取控制提供支持。这主要通过 SQL 的 GRANT 语句和 REVOKE 语句来实现。

用户权限是由两个要素组成的: 数据库对象和操作类型。定义一个用户的存取权限就是要定义这个用户可以在哪些数据库对象上进行哪些类型的操作。在数据库系统中, 定义存取权限称为授权(authorization)。

在非关系数据库系统中, 用户只能对数据进行操作, 存取控制的数据库对象也仅限于数据本身。

在关系数据库系统中, 存取控制的对象不仅有数据本身(基本表和视图中的数据、属性列上的数据), 还有数据库与模式、基本表、视图和索引的创建等。表 4.4 列出了不同对象具有的主要存取权限。

表 4.4 关系数据库系统中不同对象具有的主要存取权限

对象类型	对象	操作类型
数据库和模式	数据库和模式	CREATE
	基本表	CREATE TABLE, ALTER TABLE
	视图	CREATE VIEW
	索引	CREATE INDEX
数据	基本表和视图	SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES
	属性列	SELECT, INSERT, UPDATE, REFERENCES, ALL PRIVILEGES

表 4.4 中, 创建数据库的权限在 SQL 标准中没有给出明确定义, 因此不同的关系数据库管理系统实现的语句和语法不同。4.2.4 小节将介绍 KingbaseES 创建数据库的语句。

具有创建数据库权限的用户可以在该数据库上创建模式。

具有创建模式权限的用户可以在该模式中创建表、视图、索引等数据库对象。

表 4.4 中, 属性列权限包括 SELECT、INSERT、UPDATE、REFERENCES, 其含义与表权限类似。需要说明的是, 对列的 UPDATE 权限指对于表中存在的某一列的值可以进行修改。当然, 有了这个权限之后, 在修改的过程中还要遵守表在创建时定义的主码及其他约束。列上的 INSERT 权限指用户可以插入一个元组, 对于插入的元组, 授权用户可以插入指定的值, 其他列或者为空, 或者为默认值。在给用户授予列 INSERT 权限时, 一定要包含主码的 INSERT 权限, 否则用户的插入动作会因为主码为空而被拒绝。

4.2.4 授予与收回对数据的操作权限

SQL 中使用 GRANT 和 REVOKE 语句向用户授予或收回对数据的操作权限。GRANT 语句向用户授予权限, REVOKE 语句收回已经授予用户的权限。

1. GRANT 语句

GRANT 语句的一般格式如下:

```
GRANT <权限> [, <权限>] ...  
ON <对象类型> <对象名> [, <对象类型> <对象名>] ...  
TO <用户> [, <用户>] ...  
[WITH GRANT OPTION];
```

其语义为: 将对指定对象的指定操作权限授予指定的用户。发出该 GRANT 语句的可以是数据库管理员 (DBA), 也可以是该数据库的创建者 (即数据库属主 owner), 还可以是已经拥有该权限的用户。接受权限的用户可以是一个或多个具体用户, 也可以是 PUBLIC, 即全体用户。

如果指定了 **WITH GRANT OPTION** 子句, 则获得某种权限的用户还可以把这种权限再授予其他的用户。如果没有指定该子句, 则获得某种权限的用户只能使用该权限, 不能传播。

SQL 标准允许具有 WITH GRANT OPTION 的用户把相应权限或其子集传递授予其他用户, 但不允许循环授权, 即被授权者不能把权限再授回给授权者或其祖先, 如图 4.3 所示。



图 4.3 SQL 标准不允许循环授权

[例 4.1] 把查询 Student 表的权限授给用户 U1。

```
GRANT SELECT  
ON TABLE Student  
TO U1;
```

[例 4.2] 把对 Student 表和 Course 表的全部操作权限授予用户 U2 和 U3。

```
GRANT ALL PRIVILEGES  
ON TABLE Student, Course  
TO U2, U3;
```

[例 4.3] 把对表 SC 的查询权限授予所有用户。

```
GRANT SELECT
```

```
ON TABLE SC  
TO PUBLIC;
```

[例 4.4] 把查询 Student 表和修改学生学号的权限授予用户 U4。

```
GRANT UPDATE(Sno),SELECT  
ON TABLE Student  
TO U4;
```

这里,实际上要授予 U4 用户的是对基本表 Student 的 SELECT 权限和对属性列 Sno 的 UPDATE 权限。对属性列授权时必须明确指出相应的属性列名。

[例 4.5] 把对表 SC 的 INSERT 权限授予 U5 用户,并允许将此权限再授予其他用户。

```
GRANT INSERT  
ON TABLE SC  
TO U5  
WITH GRANT OPTION;
```

执行此 SQL 语句后, U5 不仅拥有了对表 SC 的 INSERT 权限,还可以传播此权限,即由 U5 用户发上述 GRANT 命令给其他用户。

[例 4.6] 用户 U5 将对表 SC 的 INSERT 权限授予用户 U6。

```
GRANT INSERT  
ON TABLE SC  
TO U6  
WITH GRANT OPTION;
```

同样, U6 还可以将此权限继续授予 U7。

[例 4.7] 用户 U6 将对表 SC 的 INSERT 权限授予用户 U7。

```
GRANT INSERT  
ON TABLE SC  
TO U7;
```

因为 U6 未给 U7 传播的权限,因此 U7 不能再传播此权限。

由上面的例子可以看到, GRANT 语句可以一次向一个用户授权,如例 4.1 所示,这是最简单的一种授权操作;也可以一次向多个用户授权,如例 4.2、4.3 所示;还可以一次传播多个同类对象的权限,如例 4.2 所示;甚至一次可以完成对基本表和属性列这些不同对象的授权,如例 4.4 所示。表 4.5 是执行了例 4.1~4.7 的语句后“学生选课”数据库中的用户权限定义表。

表 4.5 执行了例 4.1~4.7 语句后“学生选课”数据库中的用户权限定义

授权用户名	被授权用户名	数据库对象名	允许的操作类型	能否转授权
DBA	U1	关系 Student	SELECT	不能
DBA	U2	关系 Student	ALL	不能
DBA	U2	关系 Course	ALL	不能
DBA	U3	关系 Student	ALL	不能
DBA	U3	关系 Course	ALL	不能
DBA	PUBLIC	关系 SC	SELECT	不能
DBA	U4	关系 Student	SELECT	不能
DBA	U4	属性列 Student. Sno	UPDATE	不能
DBA	U5	关系 SC	INSERT	能
U5	U6	关系 SC	INSERT	能
U6	U7	关系 SC	INSERT	不能

2. REVOKE 语句

授予用户的权限可以由数据库管理员或其他授权者用 REVOKE 语句收回。REVOKE 语句的一般格式如下：

```
REVOKE <权限> [, <权限>] ...
ON <对象类型> <对象名> [, <对象类型> <对象名>] ...
FROM <用户> [, <用户>] ... [CASCADE|RESTRICT];
```

若语句指定了 CASCADE，则级联收回授予的权限；若语句指定了 RESTRICT，则转授权限后不能收回。默认值为 RESTRICT。

[例 4.8] 把用户 U4 修改学生学号的权限收回。

```
REVOKE UPDATE(Sno)
ON TABLE Student
FROM U4;
```

[例 4.9] 收回所有用户对表 SC 的查询权限。

```
REVOKE SELECT
ON TABLE SC
FROM PUBLIC;
```

[例 4.10] 把用户 U5 对 SC 表的 INSERT 权限收回。

```
REVOKE INSERT
```

```
ON TABLE SC
FROM U5 CASCADE;
```

执行该语句后，收回了用户 U5 的 INSERT 权限，同时级联收回了 U6 和 U7 的 INSERT 权限。因为在例 4.6 和例 4.7 中，U5 将对 SC 表的 INSERT 权限授予了 U6，而 U6 又将其授予了 U7。

注意：如果 U6 或 U7 还从其他用户处获得了对 SC 表的 INSERT 权限，则他们仍具有此权限，系统只收回直接或间接从 U5 处获得的权限。

表 4.6 是执行了例 4.8~4.10 的语句后“学生选课”数据库的用户权限定义。

表 4.6 执行了例 4.8~4.10 语句后“学生选课”数据库的用户权限定义

授权用户名	被授权用户名	数据库对象名	允许的操作类型	能否转授权
DBA	U1	关系 Student	SELECT	不能
DBA	U2	关系 Student	ALL	不能
DBA	U2	关系 Course	ALL	不能
DBA	U3	关系 Student	ALL	不能
DBA	U3	关系 Course	ALL	不能
DBA	U4	关系 Student	SELECT	不能

SQL 提供了非常灵活的授权机制。数据库管理员拥有对数据库中所有对象的所有权限，并可以根据实际情况将不同的权限授予不同的用户。

用户对自己建立的基本表和视图拥有全部的操作权限，并且可以用 GRANT 语句把其中某些权限授予其他用户。被授权的用户如果获得 WITH GRANT OPTION，还可以把获得的权限再授予其他用户。

所有授予出去的权力在必要时又都可以用 REVOKE 语句收回。

可见，用户可以“自主”地决定将数据的存取权限授予何人，并决定是否也将“授权”的权限授予别人。因此，称这样的存取控制是自主存取控制。

3. 创建数据库的权限

上面介绍的 GRANT 和 REVOKE 语句是向用户授予或收回对数据库中数据的操作权限。这里简单介绍一下表 4.4 中关于创建数据库的语句。因为创建数据库也需要进行安全控制，也是要授权的。但是在 SQL 标准中没有创建数据库的标准定义，因此各个数据库系统实现的语句不同。下面仅以金仓数据库 KingbaseES 为例做简要说明。

CREATE USER 语句建立超级用户，再由超级用户创建具有 CREATE 数据库权限的用户。

CREATE USER 语句一般语法格式如下：

```
CREATE USER <username> [ WITH ]
[ SUPERUSER | CREATEDB ] [ PASSWORD 'password';
```

具有 SUPERUSER 权限的用户是系统的超级用户，在系统中跳过权限检查，可以执行任何操作。

具有 CREATEDB 权限的用户可以创建数据库，成为数据库的属主，具有在数据库上创建模式的权限，并可以把这些权限授予其他用户。

注意，超级用户是系统初始化时指定的。例如，在安装系统时可以指定一个名称为 system 的超级用户。

[例 4.11] 创建超级用户 system2。

首先以超级用户 system 登录，然后创建 system2：

```
CREATE USER system2 WITH SUPERUSER PASSWORD '123456';
```

[例 4.12] 创建具有 CREATEDB 权限的用户 U1 和普通用户 U2。

以超级用户 system 登录，创建用户 U1、U2 如下：

```
CREATE USER U1 WITH CREATEDB PASSWORD '123456';  
/* U1 具有创建数据库的权限了 */  
CREATE USER U2 PASSWORD '123456';      /* U2 是普通用户 */
```

[例 4.13] 创建数据库 U1DB。

以 U1 用户登录，创建数据库 U1DB：

```
CREATE DATABASE U1DB;      /* U1 创建了数据库 */
```

U1 成为数据库 U1DB 的属主，它可以在 U1DB 数据库上创建 SCHEMA。

4.2.5 数据库角色

数据库角色是被命名的一组与数据库操作相关的权限，角色是权限的集合。因此，可以为具有相同权限的用户创建一个角色。使用角色来管理数据库权限可以简化授权的过程。

在 SQL 中首先用 CREATE ROLE 语句创建角色，然后用 GRANT 语句给角色授权，用 REVOKE 语句收回授予角色的权限。

1. 角色的创建

创建角色的 SQL 语句格式如下：

```
CREATE ROLE <角色名>
```

刚刚创建的角色是空的，没有任何内容。可以用 GRANT 为角色授权。

2. 给角色授权

```
GRANT <权限> [, <权限>] ...  
ON <对象类型> 对象名  
TO <角色> [, <角色>] ...
```

数据库管理员和用户可以利用 GRANT 语句将权限授予某一个或几个角色。

3. 将一个角色授予其他的角色或用户

```
GRANT <角色 1> [, <角色 2>] ...  
TO <角色 3> [, <用户 1>] ...  
[WITH ADMIN OPTION]
```

该语句把角色授予某用户,或授予另一个角色。这样,一个角色(例如角色 3)所拥有的权限就是授予它的全部角色(例如角色 1 和角色 2)所包含的权限的总和。

授予者可以是角色的创建者,或者是在这个角色上的 WITH ADMIN OPTION 权限拥有者。

如果指定了 WITH ADMIN OPTION 子句,则获得某种权限的角色或用户还可以把这种权限再授予其他的角色。

一个角色包含的权限包括直接授予这个角色的全部权限,再加上其他角色授予这个角色的全部权限。

4. 角色权限的收回

```
REVOKE <权限> [, <权限>] ...  
ON <对象类型> <对象名>  
FROM <角色> [, <角色>] ...
```

用户可以收回角色的权限,从而修改角色拥有的权限。

REVOKE 动作的执行者可以是角色的创建者,或者是在这个(些)角色上的 WITH ADMIN OPTION 权限拥有者。

[例 4.14] 通过角色来实现将一组权限授予一个用户。

步骤如下:

① 创建一个角色 R1。

```
CREATE ROLE R1;
```

② 使用 GRANT 语句,使角色 R1 拥有 Student 表的 SELECT、UPDATE、INSERT 权限。

```
GRANT SELECT, UPDATE, INSERT  
ON TABLE Student  
TO R1;
```

③ 将这个角色授予王平、张明、赵玲,使他们具有角色 R1 所包含的全部权限。

```
GRANT R1  
TO 王平, 张明, 赵玲;
```

④ 一次性通过 R1 来收回王平的这三个权限。


```
REVOKE R1  
FROM 王平;
```

[例 4.15] 给角色增加新的权限。

```
GRANT DELETE  
ON TABLE Student  
TO R1
```

使角色 R1 在原来的基础上增加了 Student 表的 DELETE 权限。

[例 4.16] 减少角色的权限。

```
REVOKE SELECT  
ON TABLE Student  
FROM R1;
```

使 R1 减少了 SELECT 权限。

可以看出, 数据库角色是一组权限的集合。使用角色来管理数据库权限可以简化授权的过程, 使自主授权的执行更加灵活、方便。

4.2.6 强制存取控制方法

自主存取控制能够通过授权机制有效地控制对敏感数据的存取。但是由于用户对数据的存取权限是“自主”的, 用户可以自由地决定将数据的存取权限授予何人, 以及决定是否也将“授权”的权限授予别人。在这种授权机制下, 仍可能存在数据的无意泄露。比如, 甲将自己权限范围内的某些数据存取权限授权给乙, 甲的意图是仅允许乙本人操纵这些数据。但甲的这种安全性要求并不能得到保证, 因为乙一旦获得了对数据的权限, 就可以将数据备份, 获得自身权限内的副本, 并在不征得甲同意的前提下传播副本。造成这一问题的根本原因就在于这种机制仅仅通过对数据的存取权限来进行安全控制, 而数据本身并无安全性标记。要解决这一问题, 就需要对系统控制下的所有主客体实施强制存取控制策略。

所谓强制存取控制, 是指系统为保证更高层次的安全性, 按照 TCSEC/TDI 标准中安全策略的要求所采取的强制存取检查手段。它不是用户能直接感知或进行控制的。强制存取控制适用于那些对数据严格且固定密级分类的部门, 例如军事部门或政府部门。

在强制存取控制方法中, 数据库管理系统所管理的全部实体被分为主体和客体两大类。

主体是系统中的活动实体, 既包括数据库管理系统所管理的实际用户, 也包括代表用户的各进程。**客体**是系统中的被动实体, 是受主体操纵的, 包括文件、基本表、索引、视图等。对于主体和客体, 数据库管理系统为它们的每个实例(值)指派一个敏感度标记(label)。

敏感度标记被分成若干级别, 例如绝密级(top secret, TS)、机密级(secret, S)、秘密级(confidential, C)、公开(public, P)等。密级的次序是 $TS \geq S \geq C \geq P$ 。主体的敏感度标记称为许可证级别, 客体的敏感度标记称为密级。强制存取控制机制就是通过对比主体和客体

的 label, 最终确定主体是否能够存取客体。

当某一用户(或主体)以标记 label 注册入系统时, 系统要求他对任何客体的存取必须遵循如下规则:

- ① 仅当主体的许可证级别大于或等于客体的密级时, 该主体才能读取相应的客体。
- ② 仅当主体的许可证级别小于或等于客体的密级时, 该主体才能写相应的客体。

这些系统规定: 仅当主体的许可证级别小于或等于客体的密级时, 该主体才能写相应的客体, 即用户可以为写入的数据对象赋予高于自己许可证级别的密级。这样一旦数据被写入, 该用户自己也不能再读该数据对象了。

规则①的含义是明显的, 而规则②需要解释一下。按照规则②, 用户可以为写入的数据对象赋予高于自己的许可证级别的密级。这样一旦数据被写入, 该用户自己也不能再读该数据对象了。如果违反了规则②, 就有可能把数据的密级从高流向低, 造成数据的泄漏。

例如, 某个绝密级的用户把一个绝密级别的数据恶意地降低为公开级别, 然后把它写回, 这样原来是绝密的数据大家都可以读到了, 造成了绝密级数据的泄漏。

这两种规则的共同点在于, 它们均禁止了拥有高许可证级别的主体更新低密级的数据对象, 从而防止了敏感数据的泄漏。

强制存取控制是对数据本身进行密级标记, 无论数据如何复制, 标记与数据是一个不可分的整体。只有符合密级标记要求的用户才可以操纵数据, 从而提供了更高级别的安全性。

前面已经提到, 较高安全性级别提供的安全保护包含较低级别的所有保护, 因此在实现强制存取控制时要首先实现自主存取控制, 即自主存取控制与强制存取控制共同构成数据库管理系统的安全机制。图 4.4 为数据库管理系统的安全检查示意图, 系统首先进行自主存取控制(DAC)检查, 对通过 DAC 检查的允许存取的数据对象, 再由系统自动进行强制存取控制(MAC)检查, 只有通过 MAC 检查的数据对象方可存取。



图 4.4 数据库管理系统的安全检查示意图

4.3 视图机制

前已提及, 视图具有安全保护的功能。为不同的用户定义不同的视图, 就可以把数据对象限制在一定的范围内, 通过视图机制把保密的数据对无权存取的用户隐藏起来, 从而自动对数据提供一定程度的安全保护。

视图机制还可以和授权机制相配合, 间接地实现支持存取谓词的用户权限定义。例如, 假定王平老师只能查询计算机科学与技术专业学生的信息, 专业负责人张明老师具有查询和增删改该专业学生信息的所有权限。这就要求系统能支持存取谓词的用户权限定义。在不直接支持存取谓词的系统中, 可以先建立计算机科学与技术专业学生的视图 CS_Student, 然后在视图上

进一步定义存取权限。

[例 4.17] 建立计算机科学与技术专业学生的视图, 把对该视图的 SELECT 权限授予王平, 把该视图上的所有操作权限授予张明。

```
CREATE VIEW CS_Student    /* 先建立视图 CS_Student */
AS
SELECT *
FROM Student
WHERE Smajor='计算机科学与技术'
WITH CHECK OPTION;      /* 对该视图进行增、删、改时, 必须满足 */
                        /* Smajor='计算机科学与技术'的条件 */
GRANT SELECT             /* 王平老师只能查询计算机科学与技术学生的信息 */
ON CS_Student
TO 王平;
GRANT ALL PRIVILEGES     /* 专业负责人张明具有查询和增、删、改该视图的所有权限 */
ON CS_Student
TO 张明;
```

4.4 审 计

数据库安全审计系统提供了一种事后检查的安全机制。前面讲的用户身份鉴别、存取控制是数据库安全保护的重要技术(安全策略方面), 但不是全部。审计(audit)功能就是数据库管理系统达到 C2 以上安全级别必不可少的一项指标。

任何系统的安全保护措施都不是完美无缺的, 蓄意盗窃、破坏数据的人总是想方设法打破控制。审计功能把用户对数据库的所有操作自动记录下来放入审计日志(audit log)。审计员可以利用审计日志监控数据库中的各种行为, 重现导致数据库现有状况的一系列事件, 找出非法存取数据的人、时间和内容等; 还可以通过对审计日志分析, 对潜在的威胁提前采取措施加以防范。

审计通常是很费时间和空间的, 所以数据库管理系统往往都将审计设置为可选特征, 允许数据库管理员根据具体应用对安全性要求灵活地打开或关闭审计功能。审计功能主要用于安全性要求较高的部门。它能对普通和特权用户行为、各种表操作、身份鉴别、自主和强制访问控制等操作进行审计。它既能审计成功操作, 也能审计失败操作。

1. 审计事件

审计事件一般有多种类别, 例如:

① 服务器事件。审计数据库服务器发生的事件, 包含数据库服务器的启动、停止、数据库服务器配置文件的重新加载等。

② 系统权限。审计对系统拥有的结构或模式对象进行的操作，要求该操作的权限是通过系统权限获得的。

③ 语句事件。审计 SQL 语句，如 DDL、DML 及 DCL 语句。

④ 模式对象事件。审计特定模式对象上进行的 SELECT 或 DML 操作。模式对象包括表、视图、存储过程、函数等，但不包括依附于表的索引、约束、触发器、分区表等。

2. 审计功能

审计功能主要包括以下几方面内容：

① 基本功能。提供多种审计查阅方式：基本、可选、有限等。

② 提供多套审计规则。审计规则一般在数据库初始化时设定，以方便审计员管理。

③ 提供审计分析和报表功能。

④ 提供审计日志管理功能。主要包括为防止审计员误删审计记录，审计日志必须先转储后删除；对转储的审计记录文件提供完整性和保密性保护；只允许审计员查阅和转储审计记录，不允许任何用户新增和修改审计记录等。

系统提供查询审计设置及审计记录信息的专门视图。对于系统权限级别、语句级别及模式对象级别的审计记录，也可通过相关的系统表直接查看。

3. AUDIT 语句和 NOAUDIT 语句

AUDIT 语句用来设置审计功能，NOAUDIT 语句则用来取消审计功能。

审计一般可以分为用户级审计和系统级审计。用户级审计是任何用户可设置的审计，主要是用户针对自己创建的数据库表或视图进行审计，记录所有用户对这些表或视图的一切成功和（或）不成功的访问要求，以及各种类型的 SQL 操作。

系统级审计只能由数据库管理员设置，用以监测成功或失败的登录要求、监测授权和收回操作，以及其他数据库级权限下的操作。

[例 4.18] 对修改 SC 表结构或修改 SC 表数据的操作进行审计。

① 先显示当前审计开关状态。

```
SHOW AUDIT_TRAIL;
```

② 打开审计开关。

```
SET AUDIT_TRAIL TO ON;
```

③ 对 SC 表设置审计。

```
AUDIT ALTER, UPDATE ON SC BY ACCESS;
```

BY ACCESS 审计方式表示系统对每个设置的审计操作都要进行记录。BY SESSION 审计方式则表示对于每次会话中涉及的同类审计操作，系统只记录最早的一次。

[例 4.19] 取消对 SC 表的 ALTER 和 UPDAE 操作审计。

NOAUDIT ALTER, UPDATE ON SC;

审计设置以及审计日志一般都存储在数据字典中。必须把审计开关打开,才可以在系统表 SYS_AUDITTRAIL 中查看到审计信息。

安全审计机制将特定用户或特定对象相关的操作记录到系统审计日志中,作为后续对操作的查询分析和追踪的依据。通过审计机制,可以约束用户可能的恶意操作。

4.5 数据加密

数据库的安全性归根结底是要保护数据的安全,特别是高度敏感性数据,例如财务数据、军事数据、国家机密数据等。除前面介绍的安全性措施外,还可以采用数据加密技术。数据加密是防止数据库数据在存储和传输中失密的有效手段。加密的基本思想是根据一定的算法将原始数据——明文(plaintext)经过一系列复杂的计算,变换为不可直接识别的格式——密文(ciphertext),从而使不知道解密算法的人无法获知数据的内容。

数据加密主要包括存储加密和传输加密。

1. 存储加密

存储加密一般有透明和非透明两种方式。透明存储加密是内核级加密保护方式,对用户完全隐蔽;非透明存储加密则是通过多个加密函数实现的。

透明存储加密是数据在写到磁盘时对数据进行加密,授权用户读取数据时再对其进行解密。由于数据加密对用户隐蔽,即用户不知道数据是被加密的密文,数据库的应用程序不需要做任何修改,只需在创建表语句中说明需加密的字段即可。当对加密数据进行增加、删除、修改和查询操作时,数据库管理系统将自动对数据进行加密、解密工作。基于数据库内核的数据存储加密、解密方法性能较好,安全性较高。

2. 传输加密

数据库在使用过程中通过网络传输登录系统的信息、SQL 语句和数据,若采用明文方式传输,容易被网络中的恶意用户截获或篡改,存在安全隐患。为了保证这些信息能够安全地进行交换和传输,数据库管理系统提供了传输加密功能。常用的传输加密方式如链路加密和端到端加密。其中,链路加密对传输数据在链路层进行加密,它的传输信息由报头和报文两部分组成,前者是路由选择信息,后者则是传送的数据信息。这种方式对报文和报头均加密。相对地,端到端加密对传输数据在发送端加密、接收端解密。它只加密报文,不加密报头。与链路加密相比,它只在发送端和接收端需要密码设备,而中间节点不需要密码设备,因此它所需的密码设备数量相对较少。但这种方式不加密报头,从而容易被非法监听者发现并从中获取敏感信息。

图 4.5 是一种基于安全套接层协议(secure socket layer protocol, SSL protocol)的数据库管理系统可信传输方案。它采用的是一种端到端的传输加密方式。在这个方案中,通信双方协商建



图 4.5 基于 SSL protocol 的数据库管理系统可信传输示意图

立可信连接，一次会话采用一个密钥，传输数据在发送端加密、接收端解密，有效降低了重放攻击和恶意篡改的风险。此外，出于易用性考虑，这个方案的通信加密还对应用程序透明。它的实现思路包含以下三点。

① 确认通信双方端点的可靠性。数据库管理系统采用基于数字证书的服务器和客户端认证方式实现通信双方的可靠性确认。用户和服务器各自持有由外部数字证书认证中心 (certificate authority, CA) 或企业内建 CA 颁发的数字证书，双方在进行通信时均首先向对方提供己方证书，然后使用本地的 CA 信任列表和证书撤销列表 (certificate revocation list, CRL) 对接收到的对方证书进行验证，以确保证书的合法性和有效性，进而保证对方确系通信的端。

② 协商加密算法和密钥。确认双方端点的可靠性后，通信双方协商本次会话的加密算法与密钥。在此过程中，通信双方利用公钥基础设施 (public key infrastructure, PKI) 方式确保服务器端和客户端协商过程通信的安全可靠。

③ 可信数据传输。在加密算法和密钥协商完成后，通信双方开始进行业务数据交换。与普通通信路径不同的是，这些业务数据在被发送之前将被用某一组特定的密钥进行加密和消息摘要计算，以密文形式在网络上传输。当业务数据被接收时，需用相同的一组特定的密钥进行解密和摘要计算。所谓特定的密钥是由先前通信双方磋商决定的，为且仅为双方共享，通常称之为会话密钥。第三方即使窃取传输密文，因无会话密钥也无法识别密文信息。一旦第三方对密文进行任何篡改，就会被真实的接收方通过摘要算法识破。另外，会话密钥的生命周期仅限于本次通信，理论上每次通信所采用的会话密钥不同，因此避免了使用固定密钥而引起的密钥存储类问题。

数据加密使用已有的密码技术和算法对数据库中存储的数据和传输的数据进行保护。常见的数据加密算法可以分为对称加密算法 (如 AES 算法) 和非对称加密算法 (如 RSA 算法)。各数据库管理系统还常常自己设计和采用特定的加密算法，使加密后数据的安全性得到进一步提高。即使攻击者获取数据源文件，也很难获取原始数据。但是，数据的加密和解密过程增加了查询处理的复杂性，将使查询效率受到影响。加密数据的密钥管理和数据加密对应用程序的影响，都是数据加密过程中需要考虑和解决的问题。

4.6 其他安全性保护

为满足较高安全等级数据库管理系统的安全性保护要求,在自主存取控制和强制存取控制之外,还有推理控制以及数据库应用中的隐蔽信道和数据隐私等技术。

1. 推理控制

推理控制(inference control)处理的是强制存取控制未解决的问题。例如,利用属性之间的函数依赖关系,用户能从低安全等级信息推导出其无权访问的高安全等级信息,进而导致信息泄露。

数据库推理控制机制用来避免用户利用其能够访问的数据推知更高密级的数据,即用户利用其被允许的多次查询结果,结合相关领域背景知识以及数据之间的约束,推导出其不能访问的数据。在推理控制方面,常用的方法如基于函数依赖的推理控制和基于敏感关联的推理控制等。例如,某公司信息系统中假设姓名和职务属于低安全等级(如公开)信息,而工资属于高安全等级(如机密)信息。用户A的安全等级较低,他通过授权可以查询自己的工资、姓名、职务,及其他用户的姓名和职务。由于工资是机密信息,因此用户A不应知道其他用户的工资。但是,若用户B的职务和用户A相同,则利用函数依赖关系职务 $\rightarrow$ 工资,用户A可通过自己的工资信息(假设3 000元),推出B的工资也是3 000元,从而导致高安全等级的敏感信息泄露。

2. 隐蔽信道

隐蔽信道(covert channel)处理的内容也是强制存取控制未解决的问题。下面的例子就是利用未被强制存取控制的SQL语句执行后反馈的信息进行了间接信息传递。

一般情况下,如果INSERT语句对UNIQUE属性列写入重复值,则系统会报错且操作失败。那么,针对UNIQUE约束列,高安全等级用户(发送者)可先向该列插入(或不插入)数据,而低安全等级用户(接收者)后向该列插入相同数据。

如果插入失败,则表明发送者已向该列插入数据,此时二者约定发送者传输信息位为0;如果插入成功,则表明发送者未向该列插入数据,此时二者约定发送者传输信息位为1。通过这种方式,高安全等级用户按事先约定方式主动向低安全等级用户传输信息,使得信息流从高安全等级向低安全等级流动,从而导致高安全等级敏感信息泄露。

3. 数据隐私

随着人们对隐私的重视,数据隐私成为数据库应用中新的数据保护模式。

数据隐私是控制不愿被他人知道或他人不便知道的个人数据的能力。数据隐私范围很广,涉及数据管理中的数据收集、数据存储、数据发布等各个阶段。例如,在数据存储阶段应避免非授权的用户访问个人的隐私数据。通常可以使用数据库安全技术实现这一阶段的隐私保护。如使用自主访问控制、强制访问控制和基于角色的访问控制以及数据加密等。在数据处理阶段,需要考虑数据推理带来的隐私数据泄露。非授权用户可能通过分析多次查询的结

果,或基于完整性约束信息推导出其他用户的隐私数据。在数据发布阶段,应使包含隐私的数据发布结果满足特定的安全性标准。如发布的关系数据表首先不能包含原有表的候选码,同时还要考虑准标识符的影响。

准标识符是能够唯一确定大部分记录的属性集合。在现有安全性标准中, k 匿名(k -anonymization)标准要求每个具有相同准标识符的记录组至少包括 k 条记录,从而控制攻击者判别隐私数据所属个体的概率。还有 l 多样性(l -diversity)标准、 t 临近(t -closeness)标准等,从而使攻击者不能从发布数据中推导出额外的隐私数据。数据隐私也是当前研究的热点。

4. “三权分立”的安全管理机制

在传统的数据库安全模型中,数据库管理员通常是具有极大权限的超级用户。而不对数据库管理员进行任何权限检查,实际上为数据库中的数据安全带来了隐患。为了解决数据库管理员权限过于集中的问题,遵照国家标准《信息安全技术 数据库管理系统安全技术要求》(GB/T 20273—2019),引进了“三权分立”的安全管理机制。详细介绍可参见二维码内容。



扩展阅读:三权分立的
安全管理机制

要想万无一失地保证数据库安全,使之免于遭到任何蓄意的破坏,几乎是不可能的。但严密的安全措施将使蓄意的攻击者付出高昂的代价,从而迫使攻击者不得不放弃其破坏企图。

本章小结

随着数据库应用的日益广泛和深入,以及计算机网络的发展,数据的安全保密越来越重要。数据库管理系统是管理数据的核心,因而其自身必须具有一整套完整而有效的安全性机制。

实现数据库系统安全性的技术和方法有多种,数据库管理系统提供的安全措施主要包括用户身份鉴别、入侵检测、自主存取控制和强制存取控制技术、视图技术和审计技术、数据存储加密和传输加密等技术。本章全面讲解了这些技术。

当前,数据库新技术的发展和应用使数据库安全性面临新的挑战,数据库安全保护技术的研究、创新和产品研发没有止境。

习 题 4

1. 什么是数据库的安全性?
2. 举例说明对数据库安全性产生威胁的因素。
3. 试述实现数据库安全性控制的常用方法和技术。
4. 什么是数据库中的自主存取控制方法和强制存取控制方法?
5. 对下列两个关系模式:

学生(学号,姓名,年龄,性别,家庭住址,班级号)

班级(班级号,班级名,班主任,班长)

请用 SQL 的 GRANT 语句完成下列授权功能:

- ① 授予用户 U1 对两个表的所有权限,并可给其他用户授权。
 - ② 授予用户 U2 对“学生”表具有查看权限,对“家庭住址”具有更新权限。
 - ③ 将对“班级”表查看权限授予所有用户。
 - ④ 将对“学生”表的查询、更新权限授予角色 R1。
 - ⑤ 将角色 R1 授予用户 U1,并且 U1 可继续授权给其他角色。
6. 今有以下两个关系模式:

职工(职工号,姓名,年龄,职务,工资,部门号)

部门(部门号,名称,经理名,地址,电话号)

请用 SQL 的 GRANT 语句和 REVOKE 语句(加上视图机制)实现以下授权定义或存取控制功能:

- ① 用户王明对两个表有 SELECT 权限。
 - ② 用户李勇对两个表有 INSERT 和 DELETE 权限。
 - ③ 每个职工只对自己的记录有 SELECT 权限。
 - ④ 用户刘星对职工表有 SELECT 权限,对“工资”字段具有更新权限。
 - ⑤ 用户张新具有修改这两个表的结构权限。
 - ⑥ 用户周平具有对两个表的所有权限,并具有给其他用户授权的权限。
 - ⑦ 用户杨兰具有从每个部门职工中 SELECT 最高工资、最低工资、平均工资的权限,但不能查看每个人的工资。
7. 针对第 6 题中①~⑦的每一种情况,撤销各用户所授予的权限。
8. 理解并解释强制存取控制机制中主体、客体、敏感度标记的含义。
9. 举例说明强制存取控制机制是如何确定主体能否存取客体的。
10. 什么是数据库的审计功能,为什么要提供审计功能?

第 4 章实验 安全性控制

实验 4.1 自主存取控制实验。理解和掌握自主存取控制权限的定义和维护方法,包括定义用户、定义角色、分配权限给用户、分配权限给角色和回收权限等基本功能。使用用户名登录数据库,验证权限分配是否正确。

***实验 4.2 审计实验。**掌握数据库审计的设置和管理方法,以便监控数据库操作,维护数据库安全,包括设置数据库审计开关、审计数据库对象、审计数据库语句、查看数据库审计记录,以及验证审计设置是否生效。

参考文献 4

- [1] US Department of Defense. Department of defense trusted computer system evaluation criteria [M]. The 'Or-

ange Book' Series, 1985.

[2] NCSC. Trusted database management system interpretation(TDI)[R], 1991.

[3] GRIFFITHS P P, WADE B W. An authorization mechanism for a relational database system[J]. ACM Transactions on Database Systems, 1976, 1(3): 242-255.

文献[3]讨论了 SYSTEM R 的授权机制。关系数据库系统原型 SYSTEM R 首先提出了授权和收回权力的概念。

[4] 刘启原, 刘怡. 数据库与信息系统的的核心[M]. 北京: 科学出版社, 2000.

文献[4]系统讲解了信息系统的的核心问题, 包括安全模型、与安全有关的标准、密码学与信息保密、攻击检测技术和理论、数据库的安全性、数据仓库的安全问题、网络安全问题、Java 语言及其核心问题、信息安全和病毒问题等。

[5] 张孝. 可信 COBASE 的系统强制存取控制的设计与实现[D]. 北京: 中国人民大学, 1998.

文献[5]系统讨论了数据库系统的核心问题, 给出了可信 COBASE 的系统设计和实现技术。可信 COBASE 是 COBASE 2.0 的安全版本, 它依据 TCSEC/TDI 对 B1 级的要求进行设计, 实现了强制存取控制和安全审计。

[6] 张俊, 彭朝晖, 肖艳芹, 等. DBMS 安全性评估保护轮廓 PP 的研究与开发[C]//中国计算机学会: 第 22 届全国数据库学术会议论文集(技术报告篇), 计算机科学, 2005: 72-75, 79.

文献[6]在深入研究 CC 以及国外主流 DBMS CC 安全性评估情况的基础上, 总结出 DBMS PP 的开发原则和方法, 并初步研究和开发了一个 EAL4 级 DBMS PP。根据该 PP 对 Kingbase ES 进行安全性开发、测试和内部评估, 取得了良好的效果。

[7] 张效祥, 徐家福. 计算机科学技术百科全书[M]. 3 版. 北京: 清华大学出版社, 2018.

[8] 国家市场监督管理总局, 中国国家标准化管理委员会. 信息安全技术数据库管理系统安全技术要求: GB/T 20273-2019[S]. 北京: 中国标准出版社, 2019.

本章知识点讲解微视频:



用户身份鉴别



权限的表达与检查



强制访问控制

数据库的完整性是指数据库数据的正确性 (correctness) 和相容性 (compatibility)。数据的正确性是指数据库数据符合现实世界语义且反映当前的实际状况。数据的相容性是指数据库同一对象在不同关系表中的数据是相同的, 一致的。

本章导读

本章系统地讲解关系数据库管理系统完整性实现的机制, 包括完整性约束的定义机制、检查方法和违约处理方法等。

实体完整性和参照完整性是关系系统的两个不变性, 即关系模型必须具备的完整性约束。其他完整性约束则可以归入用户定义的完整性, 由现实世界语义约束确定。

完整性约束作为数据库模式的一部分存入数据字典, 在数据库数据被修改时, 关系数据库管理系统的完整性约束检查机制将按照数据字典中定义的这些约束进行检查和处理。数据库完整性的定义可由 SQL 的数据定义语言 (DDL) 实现。

本章还讲解了触发器 (trigger), 触发器可以实施更为复杂的完整性定义、检查和违约操作, 具有更精细和更强大的数据控制能力。因为触发器规则中的动作体可以很复杂, 通常是一段过程化 SQL。

读者要通过上机实验掌握 SQL 的完整性约束定义功能、属性和元组完整性约束的定义语句, 能够验证完整性约束检查机制是否发挥作用, 掌握违约处理方法。进一步地, 要掌握触发器的概念, 并能使用触发器实现更复杂的完整性控制方法。

5.1 数据库完整性概述

数据库的完整性是指数据库数据的正确性和相容性。例如, 学生的学号必须唯一, 学生的性别只能是男或女, 百分制的课程成绩取值范围为 0~100, 学生所选的课程必须是学校开设的课程, 学生所在的院系必须是学校已成立的院系等。